Nombre: DAVID YELA || KEVIN PEREZ

MATERIA INTEGRADORA DE TELECOMUNICACIONES

Implementación de un modelo criptográfico híbrido poscuántico para dispositivos IoT de bajo costo

Diagrama de bloques, descripción de los bloques y diagramas de flujo

1) Diagrama de bloques general de la propuesta

La propuesta implementa y evalúa un esquema de establecimiento de sesiones TLS 1.3 con intercambio de claves híbrido poscuántico (X25519MLKEM768) sobre el bus interno I2C de un dispositivo IoT de bajo costo, y lo compara con el intercambio clásico X25519. El banco de pruebas está formado por un nodo sensor basado en el microcontrolador ESP32, que actúa como cliente TLS, y una unidad de borde Raspberry Pi 4, que actúa como servidor TLS y maestro del bus; un sensor INA219 mide la potencia del sistema durante cada corrida.

La Figura 1 organiza el proyecto en seis fases. Cada fase se representa como una franja que indica el objetivo específico que atiende; dentro de ella, los bloques se leen de izquierda a derecha y se agrupan en marcos según su naturaleza: **hardware** (cajas en tres dimensiones), **proceso** (rectángulos), **fuentes externas** (bordes punteados) y **resultado** (fondo gris). El analizador lógico aparece aparte, como instrumento externo, porque no forma parte del sistema evaluado. La flecha gruesa que une cada fase con la siguiente indica lo que la primera entrega a la segunda. La Tabla 1 resume las fases, su relación con los objetivos y su estado a la semana 11.

Tabla 1

Fases del proyecto, objetivos y estado a la semana 11

Fase	Objetivo	Semanas	Resultado	Estado
1. Requerimientos y diseño	1	1–2	Criterios de diseño del banco	Cumplida
2. Banco de pruebas	2	3–4	Bus verificado a 100 y 400 kbit/s; potencia de reposo	Cumplida
3. Comunicación segura sobre I2C	2	5–7	Sesiones TLS 1.3 clásica e híbrida	Cumplida
4. Validación de seguridad	3	8	Confidencialidad e integridad	Cumplida (6 de 6 ejecuciones aprobadas)
5. Evaluación experimental	3	9–10	Datos de la campaña	Cumplida (720 de 720 corridas válidas en los dos bloques)
6. Análisis de resultados	3	11–12	Recomendación e informe técnico	En curso (análisis estadístico y figuras listos; falta el informe)

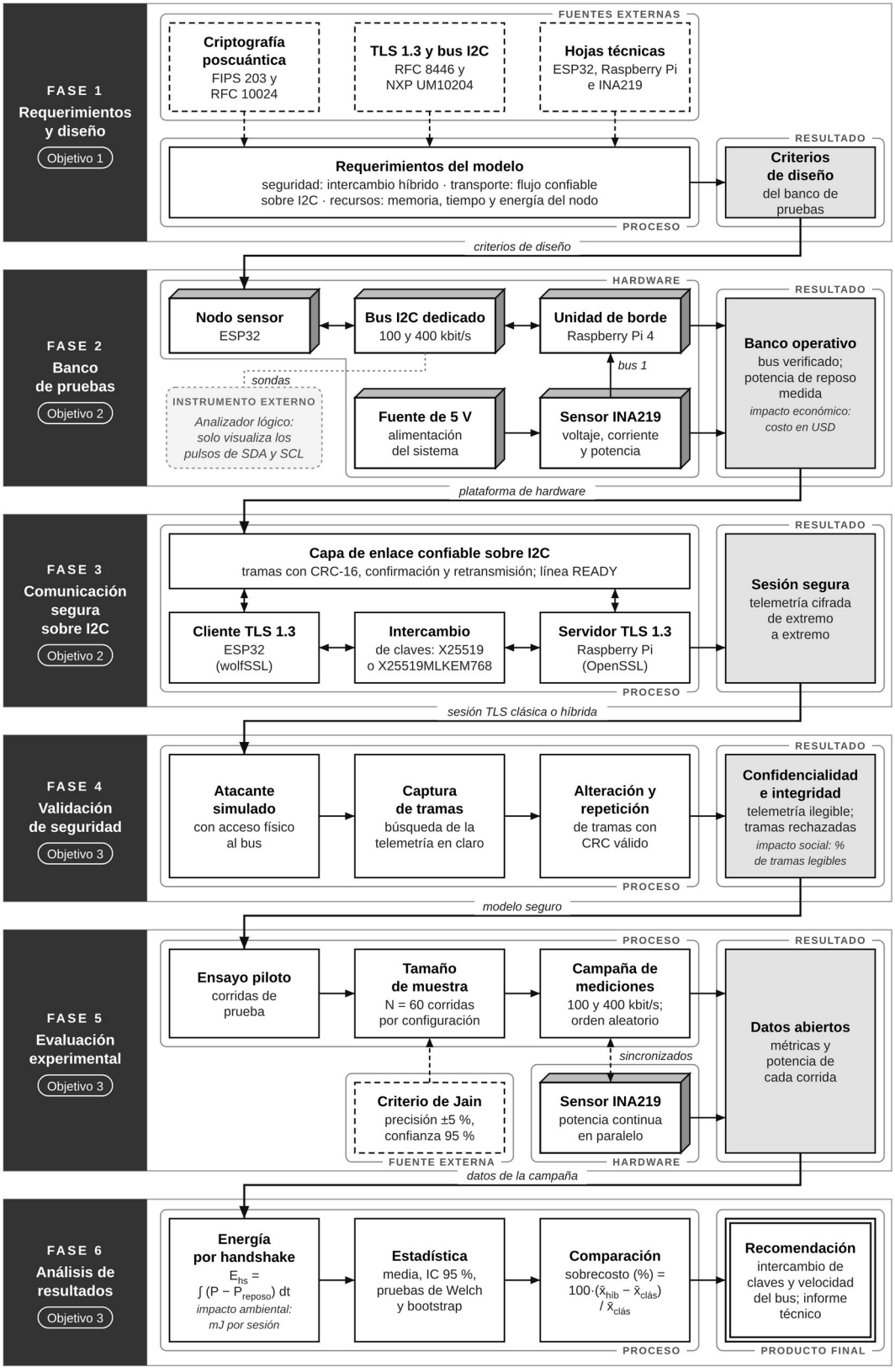
Nota. Elaboración propia a partir del cronograma de la propuesta y de las evidencias del repositorio del proyecto.

Nombre: DAVID YELA || KEVIN PEREZ

MATERIA INTEGRADORA DE TELECOMUNICACIONES

Figura 1

Diagrama de bloques general de la propuesta



Nota. Elaboración propia. Los impactos económico, social y ambiental corresponden a los indicadores registrados en la plataforma de Materia Integradora.

Nombre: DAVID YELA || KEVIN PEREZ

MATERIA INTEGRADORA DE TELECOMUNICACIONES

2) Descripción de los bloques

2.1 Fase 1: requerimientos y diseño (Objetivo 1)

- **Criptografía poscuántica (FIPS 203 y RFC 10024).** Fuente de los requisitos de seguridad: define ML-KEM-768 y el grupo híbrido X25519MLKEM768 de TLS 1.3, que combina X25519 con ML-KEM-768 para que la sesión siga protegida mientras uno de los dos algoritmos no sea roto.
- **TLS 1.3 y bus I2C (RFC 8446 y NXP UM10204).** TLS 1.3 exige un transporte confiable y ordenado; el bus I2C opera a 100 o 400 kbit/s y no cifra, no autentica ni garantiza la entrega de las tramas.
- **Hojas técnicas (ESP32, Raspberry Pi e INA219).** Fijan los límites de memoria, procesamiento, direcciones y alimentación de los dispositivos.
- **Requerimientos del modelo.** Reúne los requisitos de seguridad (intercambio de claves híbrido), de transporte (flujo confiable sobre I2C) y de recursos (memoria, tiempo y energía del nodo).
- **Criterios de diseño (resultado).** Decisiones que guían el banco: tramas de tamaño fijo con verificación de errores y retransmisión, una línea de sincronización, comparación con X25519 como línea base y medición de la energía con un sensor externo.

2.2 Fase 2: banco de pruebas (Objetivo 2)

- **Nodo sensor (ESP32).** Microcontrolador de bajo costo que actúa como cliente TLS y como esclavo I2C en la dirección 0x28; genera los mensajes de telemetría de prueba.
- **Bus I2C dedicado.** Único enlace entre los dos dispositivos (bus 3 de la Raspberry Pi, GPIO4 y GPIO5, con resistencias de pull-up de 4,7 kΩ a 3,3 V), evaluado a 100 y 400 kbit/s. La línea READY (GPIO4 del ESP32 hacia GPIO17 de la Raspberry Pi) avisa cuándo la respuesta del esclavo está lista.
- **Unidad de borde (Raspberry Pi 4).** Servidor TLS y maestro del bus; ejecuta además la orquestación, el registro de potencia y el análisis.
- **Analizador lógico (instrumento externo).** No forma parte del sistema evaluado: se conecta temporalmente a SDA y SCL solo para visualizar los pulsos y comprobar la temporización del bus, sin intervenir en él.
- **Fuente de 5 V.** Alimenta la Raspberry Pi, que a su vez alimenta al ESP32 por USB, de modo que toda la corriente del sistema pasa por el sensor.
- **Sensor INA219.** Mide el voltaje y la corriente de la alimentación con un convertidor de 12 bits; se lee por el bus 1 (dirección 0x40), separado del bus del experimento.
- **Banco operativo (resultado).** Bus verificado en ambas velocidades y potencia de reposo medida. Indicador económico: costo del banco en USD frente a equipos comerciales de medición.

Nombre: DAVID YELA || KEVIN PEREZ

MATERIA INTEGRADORA DE TELECOMUNICACIONES

2.3 Fase 3: comunicación segura sobre I2C (Objetivo 2)

- **Capa de enlace confiable sobre I2C.** Divide el flujo TLS en tramas de 128 bytes (6 de cabecera, hasta 120 de carga útil y 2 de CRC-16/CCITT), numera cada trama, la confirma con un ACK y retransmite las no confirmadas mediante un esquema de parada y espera. La línea READY sincroniza al maestro con el esclavo.
- **Cliente TLS 1.3 (ESP32, wolfSSL 5.9.4).** Inicia el handshake, verifica el certificado del servidor, cifra la telemetría y mide la duración del handshake y la memoria pico que usa la biblioteca.
- **Intercambio de claves.** X25519 como línea base clásica o X25519MLKEM768 como esquema híbrido; con el híbrido, la contribución de clave del cliente pasa de 32 a 1 216 bytes y la del servidor de 32 a 1 120 bytes.
- **Servidor TLS 1.3 (Raspberry Pi, OpenSSL 3.5.7).** Responde al handshake, registra la duración de sus etapas y los bytes intercambiados, e identifica el grupo negociado.
- **Sesión segura (resultado).** Telemetría cifrada de extremo a extremo con cualquiera de los dos intercambios.

2.4 Fase 4: validación de seguridad (Objetivo 3)

- **Atacante simulado.** La Raspberry Pi simula a un atacante con acceso físico al bus, capaz de leer, alterar y reinyectar tramas.
- **Captura de tramas.** Se registran todas las tramas de una corrida sin TLS y de otra con TLS híbrido, y se busca en ellas el texto de la telemetría.
- **Alteración y repetición.** Se invierte un bit de un registro cifrado en cada sentido y se reenvía un registro ya entregado, recalculando el CRC para que la capa de enlace no detecte la manipulación.
- **Confidencialidad e integridad (resultado).** La telemetría es legible sin TLS e ilegible con TLS, y TLS rechaza las tres manipulaciones. Indicador social: porcentaje de tramas con datos legibles.

2.5 Fase 5: evaluación experimental (Objetivo 3)

- **Ensayo piloto.** Corridas de prueba por configuración para estimar la variabilidad de cada métrica.
- **Criterio de Jain (fuente externa).** Número de repeticiones $n = (100 \cdot z \cdot s / (r \cdot \bar{x}))^2$ para una precisión relativa r = ±5 % con 95 % de confianza.
- **Tamaño de muestra.** Se fijan N = 60 corridas por configuración, por encima del mínimo de 30.
- **Campaña de mediciones.** Dos bloques, uno a 100 kbit/s y otro a 400 kbit/s; dentro de cada bloque se aleatoriza el orden del intercambio (clásico o híbrido) y del tamaño del mensaje (32, 128 y 512 bytes), con ventanas de reposo de 60 s y control de temperatura.

- **Sensor INA219 (registro continuo).** Registra la potencia en un núcleo dedicado de la Raspberry Pi, sincronizado con las marcas de tiempo de cada corrida.
- **Datos abiertos (resultado).** Métricas de cada corrida (JSONL) y potencia continua (CSV), publicadas en el repositorio del proyecto.

2.6 Fase 6: análisis de resultados (Objetivo 3)

- **Energía por handshake.** $E_{hs} = \int (P - P_{reposo})\, dt$, integrada por trapecios entre el inicio y el fin de cada handshake. Indicador ambiental: milijulios por sesión.
- **Estadística.** Se descartan las corridas con error o con limitación térmica y se calculan media, desviación e intervalo de confianza del 95 % por configuración.
- **Comparación.** Sobrecosto (%) = $100 \cdot (\bar{x}_{híb} - \bar{x}_{clás}) / \bar{x}_{clás}$ en tiempo, bytes, memoria, energía, latencia y rendimiento, con pruebas de Welch y Mann-Whitney e intervalos bootstrap.
- **Recomendación (producto final).** Intercambio de claves y velocidad del bus adecuados para proteger un bus interno, documentados en el informe técnico.

3) Diagramas de flujo de los programas

Los programas se escribieron en C sobre ESP-IDF para el ESP32 y en Python para la Raspberry Pi. La Tabla 2 relaciona cada programa con el dispositivo en que se ejecuta, la fase en que interviene y la figura que contiene su diagrama de flujo. Los programas auxiliares de verificación (prueba_eco.py, sistema.py, medir_reposo.py y resumen_rapido.py) solo prueban el bus, leen el estado de la Raspberry Pi o resumen datos, por lo que no se incluyen sus diagramas. Los diagramas siguen la simbología de la norma ISO 5807: terminal (óvalo), proceso (rectángulo), subproceso (rectángulo con barras laterales), entrada o salida de datos (paralelogramo), decisión (rombo) y conector (círculo); los recuadros punteados agrupan las partes de cada programa.

Tabla 2

Programas del proyecto y diagramas de flujo correspondientes

Programa	Dispositivo	Fase	Función	Figura
main.c, cliente_tls.c	ESP32	3 y 5	Arranque del nodo y cliente TLS	2
enlace.py, enlace.c	Raspberry Pi y ESP32	3	Capa de enlace sobre I2C	3
servidor_tls.py	Raspberry Pi	3 y 5	Servidor TLS y ejecución de una corrida	4
campana.py	Raspberry Pi	5	Orquestación de la campaña de mediciones	5
potencia.py	Raspberry Pi	2 y 5	Registro de potencia con el INA219	6
ataque.py	Raspberry Pi	4	Pruebas de confidencialidad e integridad	7
analisis.py, tamano_muestra.py	Raspberry Pi	5 y 6	Energía, estadística y tamaño de muestra	8

Nota. Elaboración propia. Programas en esp32/firmware/main/ y pi/ del repositorio.

Nombre: DAVID YELA || KEVIN PEREZ

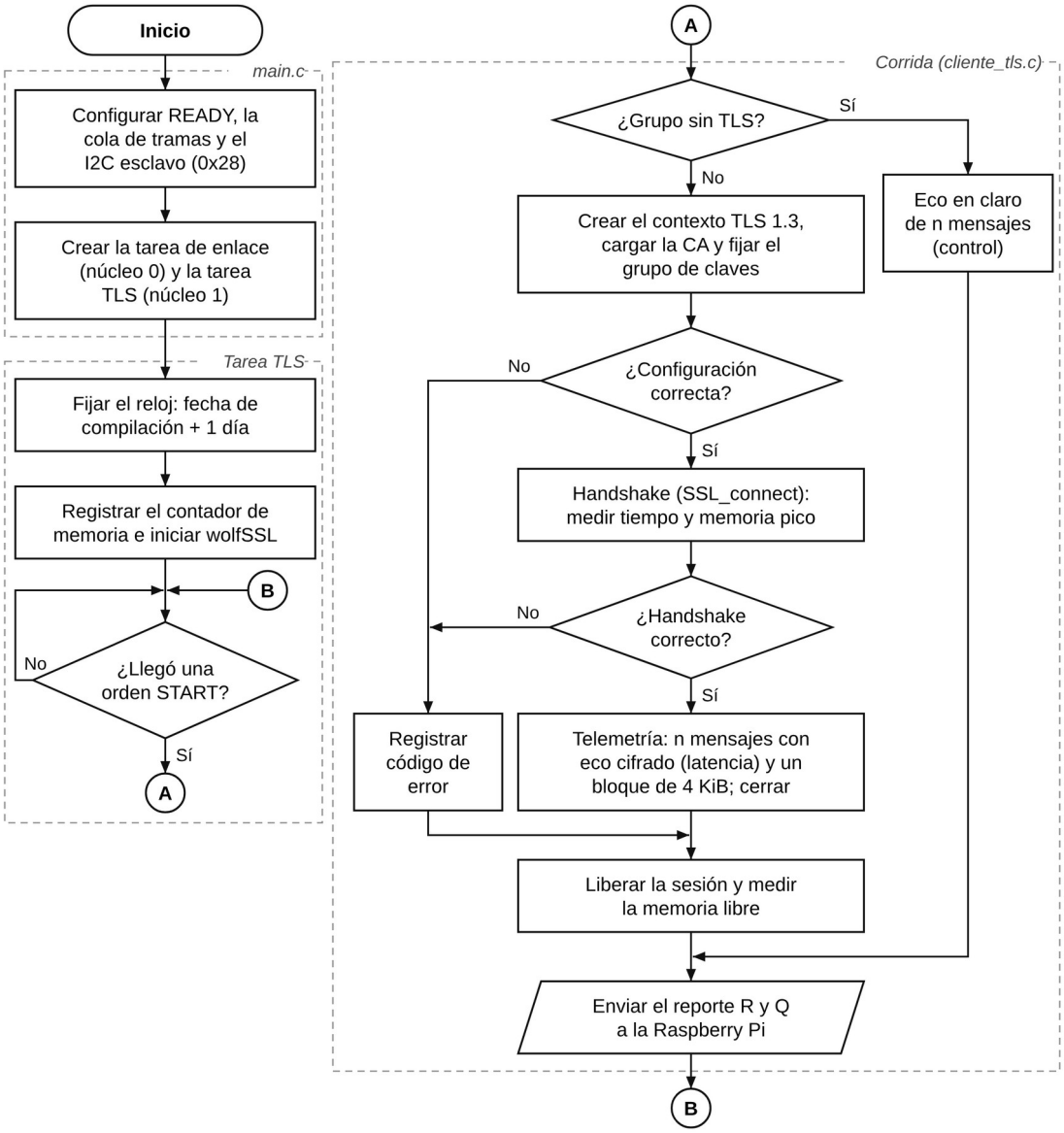
MATERIA INTEGRADORA DE TELECOMUNICACIONES

3.1 Firmware del nodo sensor (main.c y cliente_tls.c)

Al encenderse, el ESP32 configura la línea READY, la cola de tramas y el periférico I2C en modo esclavo, y crea dos tareas en núcleos distintos: la de enlace, que atiende el bus (Figura 3), y la TLS. La tarea TLS fija el reloj, necesario para validar el certificado, inicia wolfSSL y espera órdenes. Cada orden START ejecuta una corrida: en el grupo de control devuelve la telemetría en claro; en los demás crea el contexto TLS, realiza el handshake con el grupo indicado y, si tiene éxito, intercambia la telemetría y un bloque de 4 KiB. Al final envía un reporte con el código de estado, los tiempos y la memoria utilizada.

Figura 2

Diagrama de flujo del firmware del ESP32



Nota. Elaboración propia con base en el código del repositorio del proyecto.

Nombre: DAVID YELA || KEVIN PEREZ

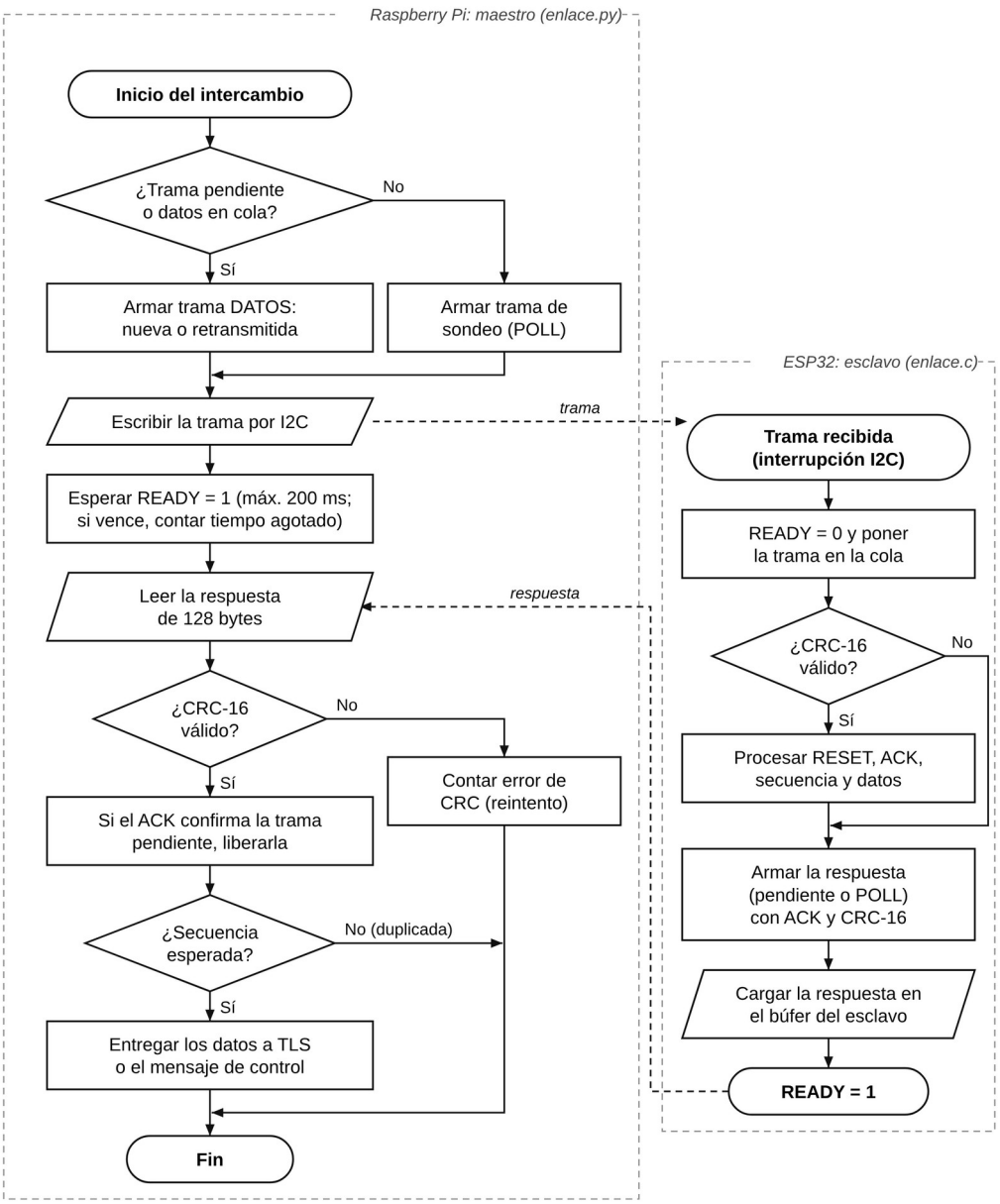
MATERIA INTEGRADORA DE TELECOMUNICACIONES

3.2 Capa de enlace sobre I2C (enlace.py y enlace.c)

La Raspberry Pi, como maestro, inicia cada intercambio: envía una trama de datos (nueva o retransmitida) o, si no tiene datos, una trama de sondeo; espera la señal READY y lee la respuesta. Las tramas con CRC incorrecto se descartan y se cuentan, el ACK recibido libera la trama pendiente y solo se entregan a TLS las tramas con la secuencia esperada, lo que elimina duplicados. En el ESP32, la interrupción del bus baja READY y encola la trama; la tarea de enlace la procesa, arma la respuesta con su CRC y vuelve a subir READY. Las líneas punteadas indican el tráfico por el bus.

Figura 3

Diagrama de flujo de la capa de enlace en el maestro y en el esclavo



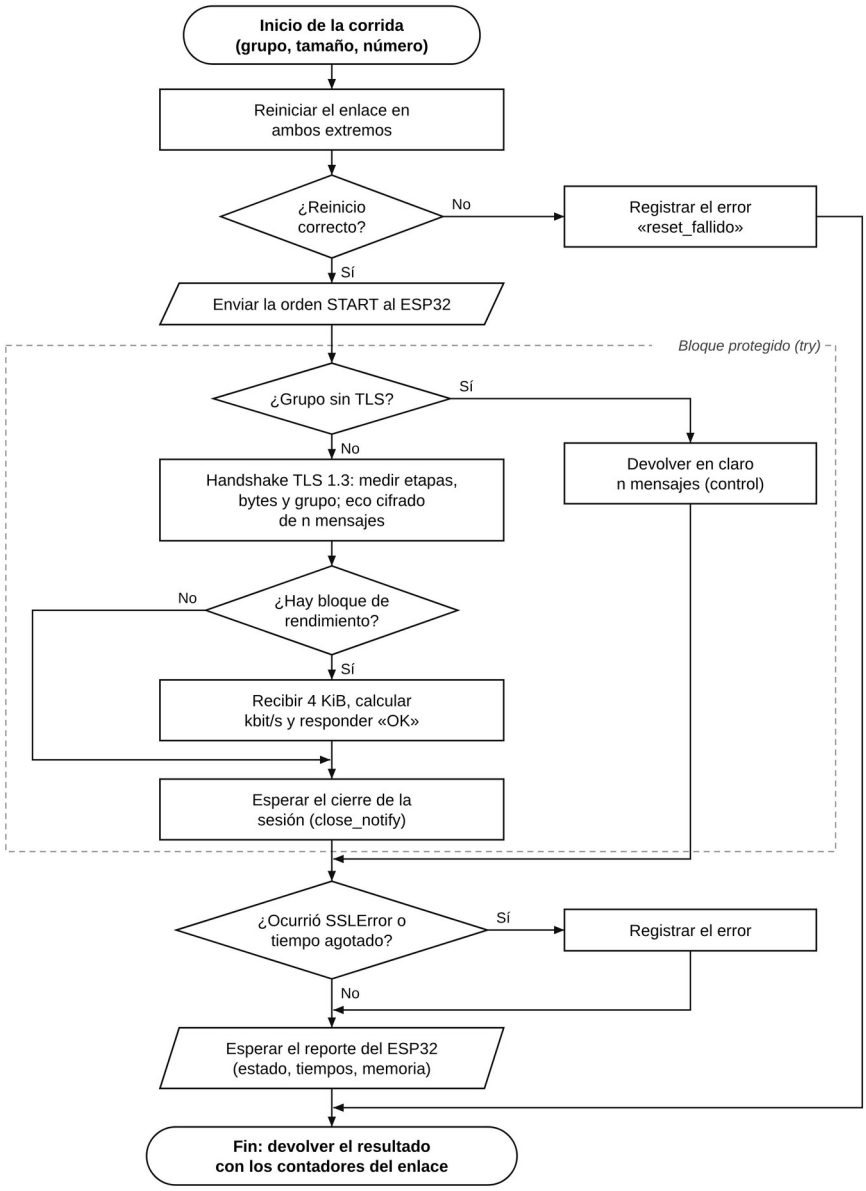
Nota. Elaboración propia con base en el código del repositorio del proyecto.

3.3 Servidor TLS y ejecución de una corrida (servidor_tls.py)

Cada corrida reinicia la capa de enlace en ambos extremos y envía la orden START. En el grupo de control los mensajes se devuelven en claro; en los demás se realiza el handshake, se registran sus etapas, los bytes y el grupo negociado, se devuelve el eco cifrado de la telemetría y, si corresponde, se recibe el bloque de rendimiento. Las excepciones de TLS o de tiempo agotado se registran como error de la corrida. Al final se espera el reporte del ESP32 y se devuelve el resultado junto con los contadores del enlace.

Figura 4

Diagrama de flujo de una corrida en el servidor TLS



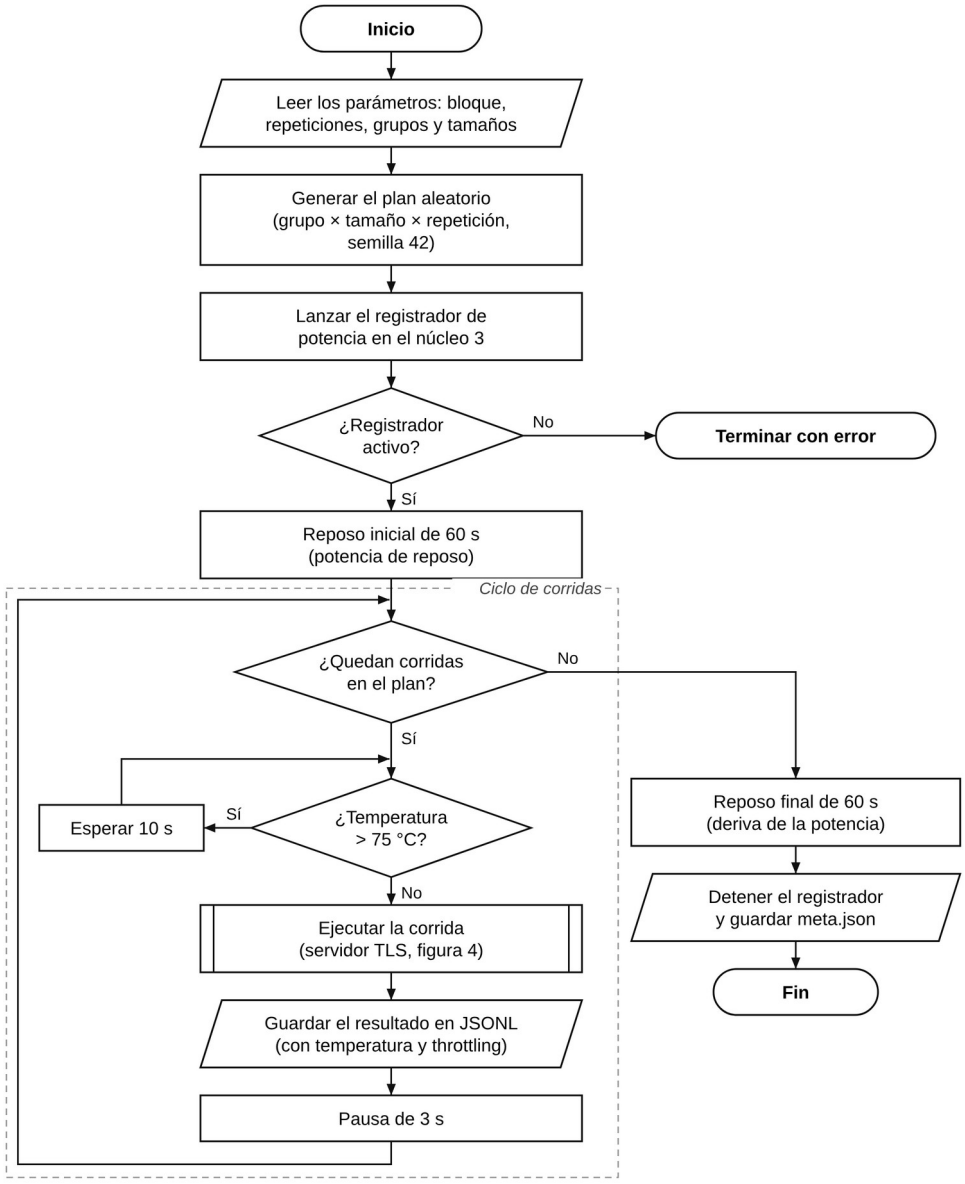
Nota. Elaboración propia con base en el código del repositorio del proyecto.

3.4 Orquestador de la campaña (campana.py)

El orquestador lee los parámetros del bloque, genera un orden aleatorio de corridas con semilla fija para que pueda reproducirse y lanza el registrador de potencia en otro núcleo. Mide 60 s de reposo, ejecuta las corridas con una pausa de 3 s entre ellas, espera si la temperatura del procesador supera 75 °C, mide otros 60 s de reposo para estimar la deriva y guarda los metadatos del bloque.

Figura 5

Diagrama de flujo del orquestador de la campaña



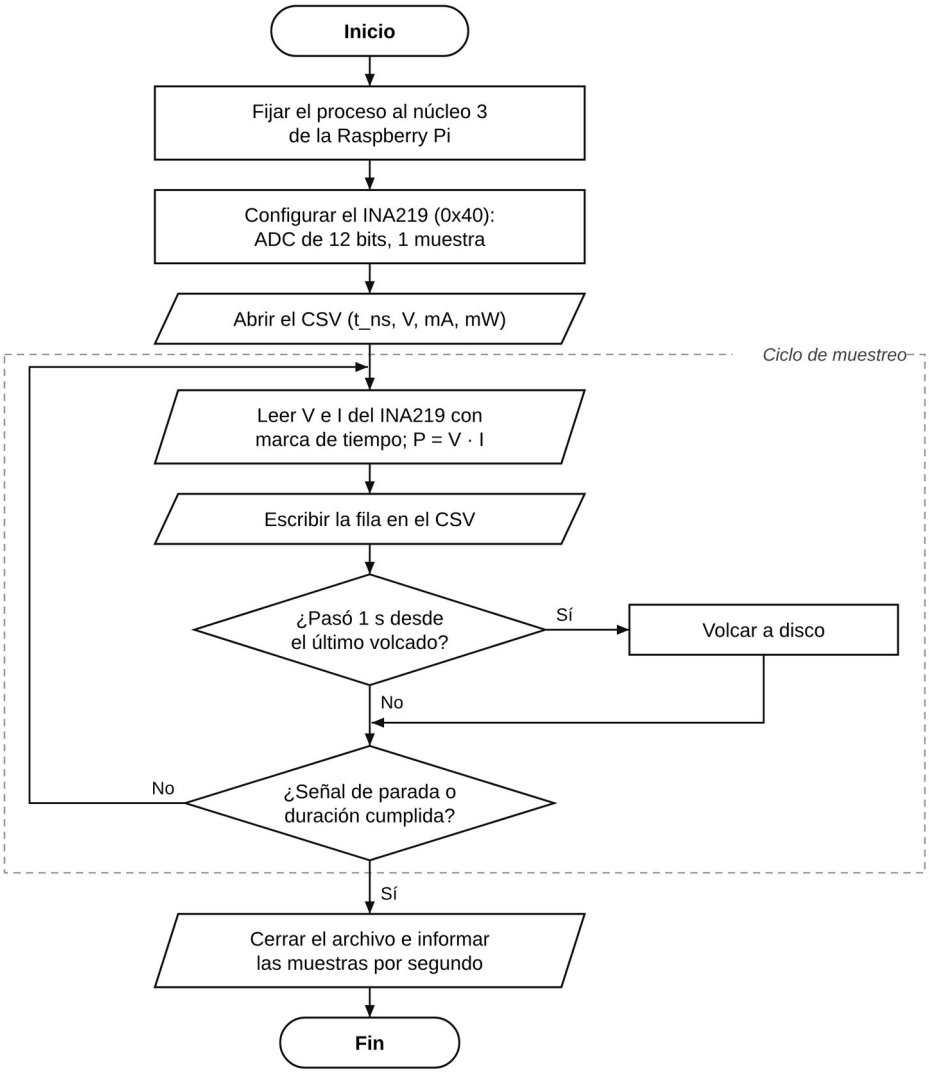
Nota. Elaboración propia con base en el código del repositorio del proyecto.

3.5 Registrador de potencia (potencia.py)

El registrador se fija a un núcleo de la Raspberry Pi para no interferir con el experimento, configura el INA219 con resolución de 12 bits y una muestra por conversión, y lee de forma continua el voltaje y la corriente, calcula la potencia y guarda cada muestra con su marca de tiempo. Vuelca el archivo a disco cada segundo y termina cuando el orquestador envía la señal de parada.

Figura 6

Diagrama de flujo del registrador de potencia



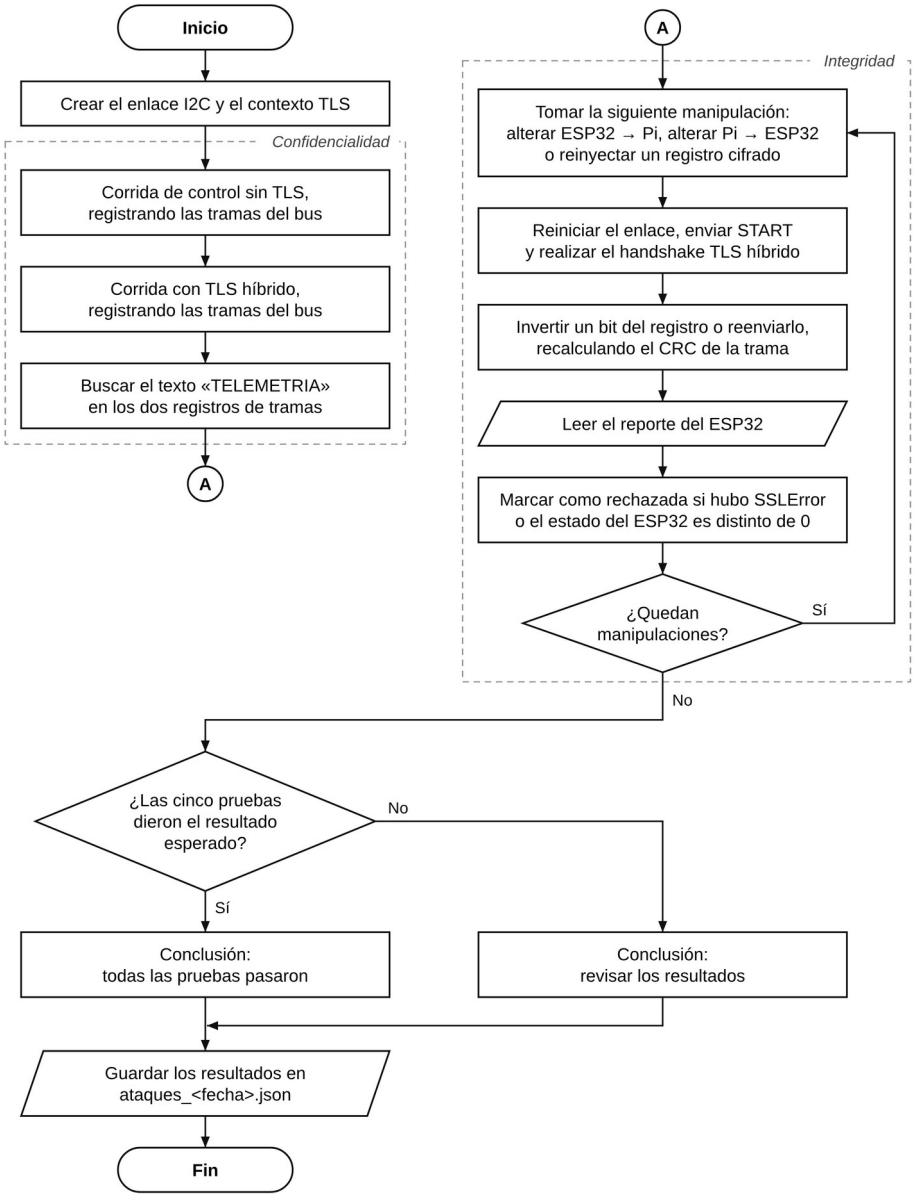
Nota. Elaboración propia con base en el código del repositorio del proyecto.

3.6 Pruebas de seguridad (ataque.py)

El programa ejecuta una corrida sin TLS y otra con TLS híbrido registrando todas las tramas del bus, y busca en ellas el texto de la telemetría. Después realiza tres manipulaciones: alterar un registro cifrado del ESP32 hacia la Raspberry Pi, alterar uno en sentido contrario y reinyectar un registro ya entregado, recalculando el CRC en cada caso. Las pruebas pasan si el texto solo es visible sin TLS y TLS rechaza las tres manipulaciones.

Figura 7

Diagrama de flujo de las pruebas de seguridad



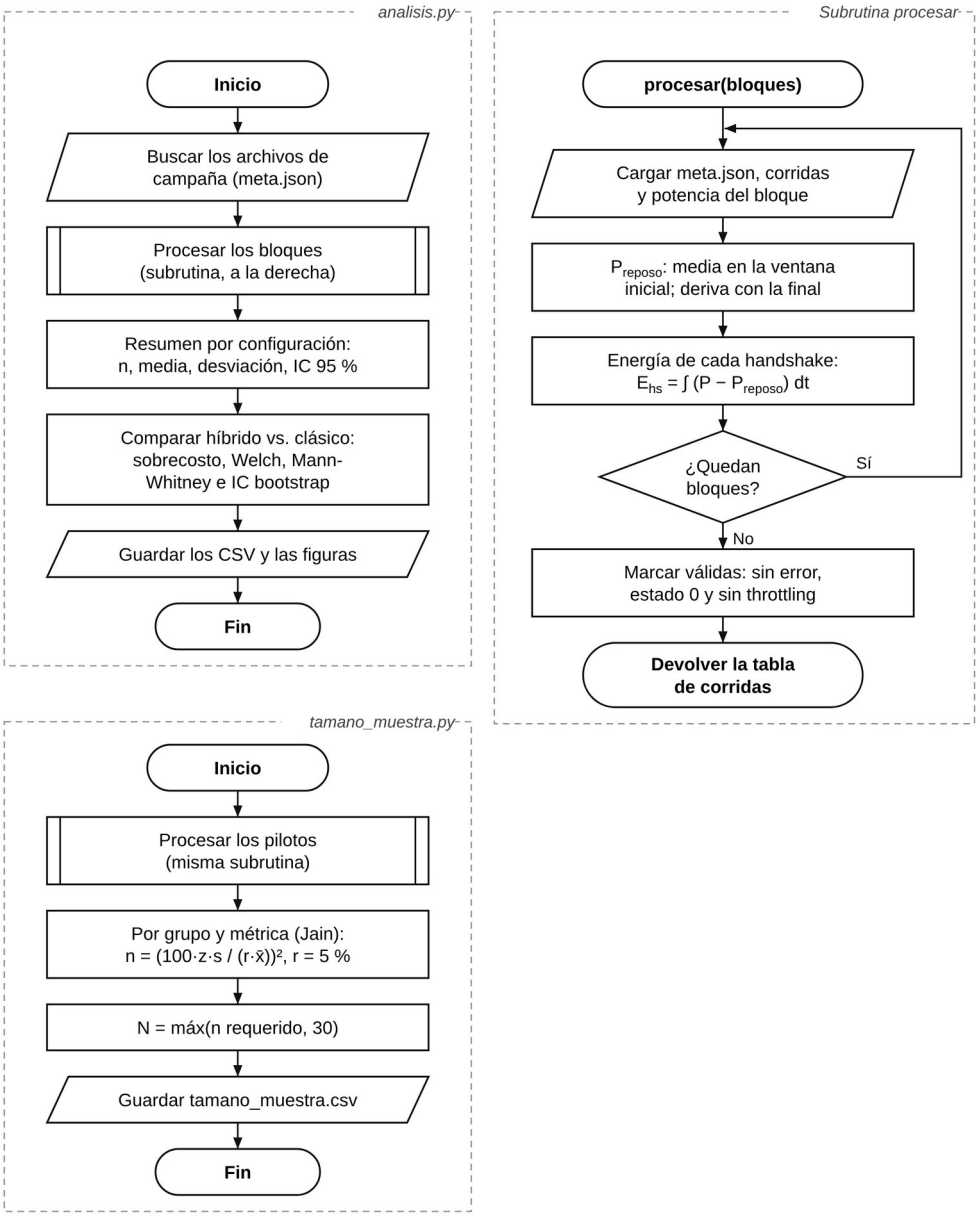
Nota. Elaboración propia con base en el código del repositorio del proyecto.

3.7 Análisis de datos y tamaño de muestra (analisis.py y tamano_muestra.py)

Ambos programas usan la subrutina procesar, que por cada bloque carga los archivos, calcula la potencia de reposo y su deriva, integra la energía de cada handshake y marca las corridas válidas. Con la campaña, analisis.py resume cada configuración y compara el intercambio híbrido con el clásico; con los ensayos piloto, tamano_muestra.py aplica el criterio de Jain para obtener el número de repeticiones, con un mínimo de 30.

Figura 8

Diagrama de flujo del análisis de datos y del cálculo del tamaño de muestra



Nota. Elaboración propia con base en el código del repositorio del proyecto.

Nombre: DAVID YELA || KEVIN PEREZ

MATERIA INTEGRADORA DE TELECOMUNICACIONES

4) Referencias

Jain, R. (1991). The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling. John Wiley & Sons.

Kwiatkowski, K., Kampanakis, P., Westerbaan, B., & Stebila, D. (2026). Post-Quantum Traditional (PQ/T) Hybrid Key Agreement Mechanisms for TLS 1.3 (RFC 10024). RFC Editor. https://www.rfc-editor.org/info/rfc10024

NIST. (2024). Module-Lattice-Based Key-Encapsulation Mechanism Standard. FIPS 203. https://doi.org/10.6028/NIST.FIPS.203

NXP Semiconductors. (2021). UM10204: I2C-bus specification and user manual (Rev. 7.0). https://www.nxp.com/docs/en/user-guide/UM10204.pdf

Rescorla, E. (2018). The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446. https://www.rfc-editor.org/info/rfc8446